

Backend Engineering Take-Home Exercise

Reliable Payment API with Idempotency

Background

You are building a payment API used by businesses to send money to recipients.

The API must safely handle retries, duplicate requests, concurrent submissions, and unreliable downstream providers.

In payment systems, clients often retry requests because they did not receive a response. A request may have been processed successfully even when the client experienced a timeout. The system must therefore ensure that retrying the same request does not result in duplicate payments.

Your task is to implement a small but production-conscious payment service.

Objective

Build an API that:

1. Accepts payment requests.
2. Prevents duplicate payments using idempotency keys.
3. Stores payments and payment attempts.
4. Integrates with a simulated external payment provider.
5. Exposes payment status.
6. Handles concurrent and repeated requests safely.

The quality of the design, reasoning, and tests is more important than the amount of code written.

Technology

Use one of the following:

- Java with Spring Boot
- Go
- Node.js with TypeScript

Use PostgreSQL as the database.

The application should be runnable locally using Docker Compose.

You may use any reasonable libraries, but be prepared to explain your choices.

Core API

1. Create a payment

```
POST /api/v1/payments
```

The idempotency key must be supplied using a request header:

```
Idempotency-Key: 4b1f35d4-3ea7-4321-a777-c06fc3389478
```

Example request:

```
{
  "organisationId": "8b24a9b4-58f5-42f1-a6ef-697cfb321164",
  "customerReference": "INV-2026-001",
  "amount": 1500.00,
  "currency": "KES",
  "recipient": {
    "type": "MOBILE_MONEY",
    "phoneNumber": "+254712345678"
  },
  "description": "Supplier payment"
}
```

A successful response should resemble:

```
{
  "paymentId": "7529bd27-a9f9-4bf0-a6b5-f9e120fda8ca",
  "status": "PENDING",
  "amount": 1500.00,
  "currency": "KES",
  "customerReference": "INV-2026-001",
  "createdAt": "2026-07-07T08:30:00Z"
}
```

A successful creation may return:

```
201 Created
```

A repeated request using the same idempotency key and identical request data must return the original payment response without creating another payment.

A repeated request using the same idempotency key but different request data must be rejected.

Suggested response:

```
409 Conflict
```

```
{
  "error": "idempotency_key_reused",
  "message": "The idempotency key has already been used with a different request"
}
```

2. Retrieve payment status

```
GET /api/v1/payments/{paymentId}
```

Example response:

```
{
  "paymentId": "7529bd27-a9f9-4bf0-a6b5-f9e120fda8ca",
  "organisationId": "8b24a9b4-58f5-42f1-a6ef-697cfb321164",
  "customerReference": "INV-2026-001",
  "amount": 1500.00,
  "currency": "KES",
  "status": "SUCCESS",
  "providerReference": "PROV-9837461",
  "createdAt": "2026-07-07T08:30:00Z",
  "updatedAt": "2026-07-07T08:30:04Z"
}
```

Payment statuses

At minimum, support:

```
PENDING
PROCESSING
SUCCESS
FAILED
```

The implementation should prevent invalid status transitions.

For example:

PENDING -> PROCESSING
PROCESSING -> SUCCESS
PROCESSING -> FAILED

A successful payment must not later become pending.

Document any additional transitions you support.

Simulated payment provider

Implement or include a simulated external provider.

The provider should expose an endpoint such as:

```
POST /provider/payments
```

Example provider request:

```
{
  "requestId": "7529bd27-a9f9-4bf0-a6b5-f9e120fda8ca",
  "amount": 1500.00,
  "currency": "KES",
  "recipient": "+254712345678"
}
```

Example provider response:

```
{
  "providerReference": "PROV-9837461",
  "status": "SUCCESS"
}
```

The provider should support configurable failure behaviour.

At minimum, it should be possible to simulate:

- Successful processing
- Rejected payment
- HTTP 500 response
- Request timeout
- Successful processing followed by a timeout

The last scenario is important.

In that scenario, the provider processes the payment successfully but the client does not receive the response.

Your application must avoid blindly creating another provider payment when the request is retried.

Idempotency requirements

The following behaviours are required.

Same key and same request

Two requests with the same:

- Organisation
- Idempotency key
- Request body

must represent the same payment.

The second request should return the existing result.

It must not create:

- Another payment
- Another provider submission
- Another successful debit

Same key and different request

The following must be rejected:

```
First request:
Idempotency-Key: abc-123
Amount: 1,000 KES

Second request:
Idempotency-Key: abc-123
Amount: 5,000 KES
```

Different organisations

The same idempotency key may be used by different organisations.

For example:

```
Organisation A + abc-123
Organisation B + abc-123
```

These should be treated as separate requests.

The uniqueness boundary should therefore include the organisation.

Concurrency requirements

Your implementation must safely handle two identical requests arriving at almost the same time.

Example:

```
Request A begins at 10:00:00.001
Request B begins at 10:00:00.002
```

Both use the same organisation and idempotency key.

Only one payment must be created.

Do not rely only on:

- An in-memory map
- A synchronized block
- Application-level checks before insert

Assume the service may eventually run on multiple replicas.

Use database constraints, transactions, locking, or another durable mechanism.

Data model

At minimum, store the following concepts.

Payment

Suggested fields:

```
id
organisation_id
idempotency_key
request_hash
customer_reference
```

```
amount
currency
recipient_type
recipient_value
description
status
provider_reference
failure_code
failure_message
created_at
updated_at
```

Payment attempt

A payment may have one or more processing attempts.

Suggested fields:

```
id
payment_id
attempt_number
provider_request_id
status
request_payload
response_payload
error_code
error_message
started_at
completed_at
created_at
```

Sensitive information should not be unnecessarily stored or logged.

You may adjust the schema where appropriate.

Validation

At minimum, validate:

- `organisationId` is required
- `customerReference` is required
- Amount must be greater than zero
- Currency is required
- Recipient type is required
- Mobile money payments require a phone number
- Idempotency key is required
- Idempotency key length is limited

- Unsupported currencies are rejected

Use appropriate HTTP status codes.

Error handling

Return structured errors.

Example:

```
{
  "error": "invalid_amount",
  "message": "Amount must be greater than zero"
}
```

Do not return internal stack traces to the client.

The application should distinguish between:

- Validation failure
 - Idempotency conflict
 - Payment not found
 - Provider rejection
 - Temporary provider failure
 - Internal failure
-

Required scenarios

Your implementation should correctly handle all the following scenarios.

Scenario 1: Normal successful payment

1. Client submits payment.
2. Payment is stored.
3. Provider is called.
4. Provider returns success.
5. Payment becomes `SUCCESS`.

Scenario 2: Client repeats the request

1. Client submits payment.
2. Payment succeeds.
3. Client retries using the same idempotency key.
4. Existing payment is returned.
5. Provider is not called again.

Scenario 3: Concurrent requests

1. Two requests arrive at the same time.
2. Both use the same organisation and idempotency key.
3. One payment is created.
4. Both callers receive the same payment result.

Scenario 4: Idempotency key reused incorrectly

1. Client submits a payment for KES 1,000.
2. Client reuses the same idempotency key for KES 2,000.
3. The second request is rejected.

Scenario 5: Provider rejects the payment

1. Provider responds with a business rejection.
2. Payment becomes **FAILED**.
3. Failure information is recorded.

Scenario 6: Provider timeout

1. Provider does not respond.
2. The system must not incorrectly mark the payment as successful.
3. The system should preserve enough information to reconcile or safely retry the payment.

Scenario 7: Provider processes payment but response is lost

1. Provider processes the request successfully.
2. The connection times out before the response reaches your service.
3. The client retries the payment request.
4. Your service must avoid creating a second provider payment.

Document how your design handles this uncertainty.

Testing requirements

Include automated tests.

At minimum, provide tests for:

- Successful payment creation
- Missing idempotency key
- Invalid amount
- Unsupported currency
- Same key and same request
- Same key and different request
- Same key used by different organisations
- Payment status lookup

- Provider rejection
- Provider timeout
- Concurrent identical requests

At least one test must exercise real concurrency rather than calling the service sequentially.

Database integration tests are strongly preferred for idempotency and concurrency behaviour.

Mock-only tests are not sufficient to prove database concurrency safety.

Observability

Add structured logs for important events such as:

- Payment created
- Existing idempotent response returned
- Provider request started
- Provider request completed
- Provider timeout
- Payment status changed

Do not log:

- Full request payloads containing personal information
- Full phone numbers
- Secrets
- Authorization headers
- Database credentials

Include basic metrics or describe which metrics you would add in production.

Useful examples include:

```
payments_created_total
payments_success_total
payments_failed_total
provider_requests_total
provider_timeouts_total
idempotency_replays_total
idempotency_conflicts_total
payment_processing_duration
```

Database migration

Include a migration that creates all required tables, constraints, and indexes.

The migration should include a database-level uniqueness constraint that supports the idempotency requirement.

For example, the design may enforce uniqueness across:

`organisation_id + idempotency_key`

Do not depend only on a repository lookup to prevent duplicates.

Deliverables

Submit:

1. Source code
2. Database migrations
3. Docker Compose configuration
4. Automated tests
5. README with setup instructions
6. API examples
7. Short technical note

The technical note should explain:

- How idempotency is implemented
 - How concurrent requests are handled
 - How request equality is determined
 - How provider timeouts are handled
 - How duplicate provider payments are prevented
 - Transaction boundaries
 - Important trade-offs
 - Known limitations
 - What you would improve for production
-

Constraints

Please keep the exercise focused.

You are not required to implement:

- Authentication
- A user interface
- Real mobile-money integrations
- Kubernetes deployment
- Full payment reconciliation
- A complete double-entry ledger

You may describe where these would fit into a larger production system.

Expected completion scope

A complete solution does not need to cover every possible production concern.

We are mainly looking for:

- Correctness
- Data integrity
- Clear reasoning
- Safe handling of retries
- Concurrency awareness
- Good tests
- Maintainable code

A smaller implementation with strong guarantees is preferable to a larger implementation with unsafe behaviour.

Evaluation criteria

Correctness and data integrity — 25%

- Duplicate payments are prevented
- Payment state remains consistent
- Database constraints are appropriate
- Monetary values are handled safely

Idempotency and concurrency — 25%

- Same requests return the same payment
- Conflicting requests are rejected
- Concurrent requests are handled safely
- The implementation works beyond a single application instance

Failure handling — 15%

- Provider failures are classified correctly
- Timeouts are handled cautiously
- Uncertain payment outcomes are considered
- Invalid transitions are prevented

Tests — 15%

- Tests cover important business scenarios
- At least one real concurrency test exists

- Integration tests exercise the database behaviour
- Tests are clear and maintainable

Code quality — 10%

- Responsibilities are separated clearly
- Naming is understandable
- Error handling is consistent
- Code is reasonably easy to extend

Documentation and reasoning — 10%

- Setup instructions work
- Trade-offs are explained
- Assumptions are stated
- Limitations are acknowledged

Follow-up discussion

After submission, be prepared to:

1. Walk through the payment lifecycle.
2. Explain how concurrent requests are controlled.
3. Demonstrate the automated tests.
4. Explain the provider timeout scenario.
5. Make a small change to the implementation.

Possible follow-up changes may include:

- Idempotency expires after 24 hours.
- Payments are processed asynchronously.
- The service runs with five replicas.
- The provider sends asynchronous callbacks.
- A payment may be retried through a second provider.
- Organisation limits must be checked before processing.
- The same customer reference must not be paid twice.

The follow-up discussion is part of the assessment.

Optional extensions

These are optional and should only be attempted after the core requirements work correctly.

- Asynchronous processing using a queue
- Provider callback endpoint
- Retry scheduling
- Dead-letter handling

- Payment reconciliation endpoint
- OpenAPI documentation
- Prometheus metrics
- Request tracing
- Optimistic locking
- Payment status history
- Provider circuit breaker
- Multiple payment providers